Turn GitHub Copilot into your ultimate Azure sidekick

June 17th 2025

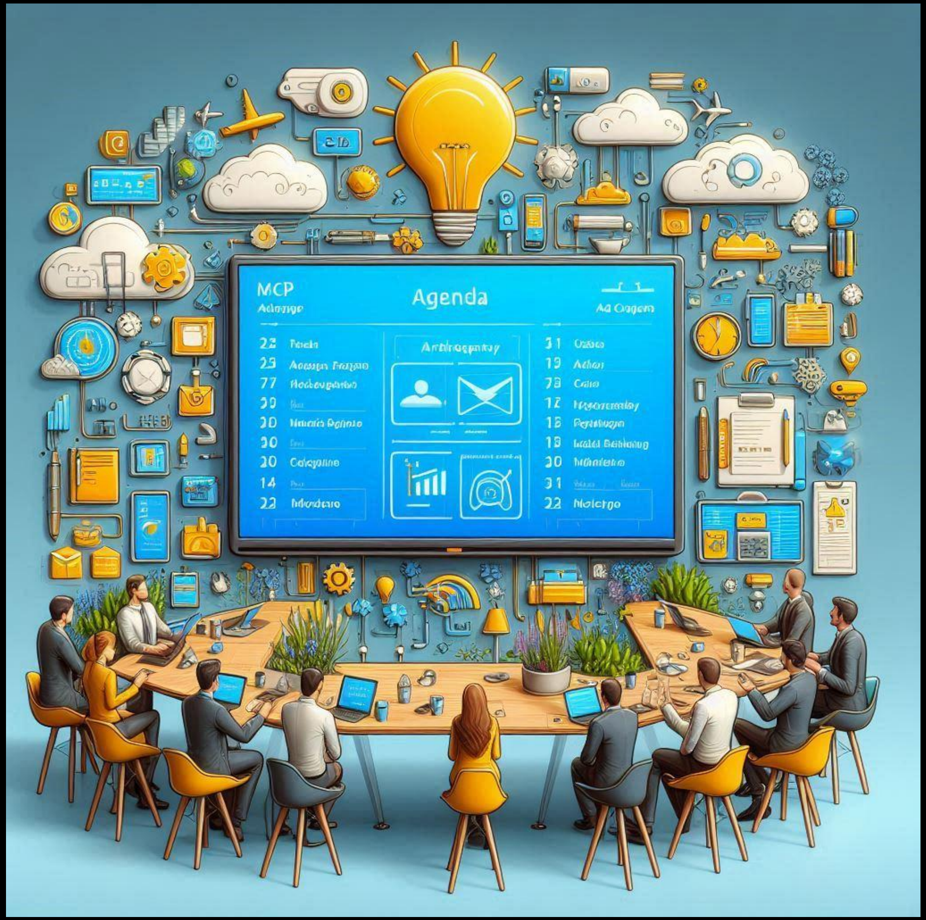
AJ Bajada

Azure, DevOps and automation enthusiast

And of course… Star Wars!

GitHub handle: tw3lveparsecs

Agenda



- MCP
- MCP Architecture
- Azure MCP Server
- Security
- Workshop

MCP

What is MCP?

MCP (Model Context Protocol) is an open protocol that standardises how AI models interact with external tools, databases, and APIs.

Think of MCP like a USB-C port for AI applications. It provides a universal way for AI models to connect to various data sources.

Why MCP Matters?

AI models often need real-time data and external tool access to enhance their responses.

MCP enables structured communication, allowing AI models to fetch live data, execute tasks, and integrate seamlessly with enterprise applications.

Key Benefits of MCP

Standardised AI-Tool Interaction: No need for custom integrations.

Real-Time Data Access: AI assistants can retrieve live information from APIs, databases, and internal systems.

Enterprise-Ready Security: Ensures safe and scalable integration with business applications.

Multi-Modal Support: Enables a unified ecosystem where text, images, audio, and potentially other data types can be processed, interpreted, and correlated in real-time.

MCP Architecture

MCP Components

MCP Hosts: Programs like Claude Desktop, IDEs, or AI tools that want to access data through MCP

MCP Clients: Protocol clients that maintain 1:1 connections with servers

MCP Servers: Lightweight programs that each expose specific capabilities through the standardised Model Context Protocol

Local Data Sources: Your computer's files, databases, and services that MCP servers can securely access

Remote Services: External systems available over the internet (e.g., through APIs) that MCP servers can connect to

Data Flow in MCP

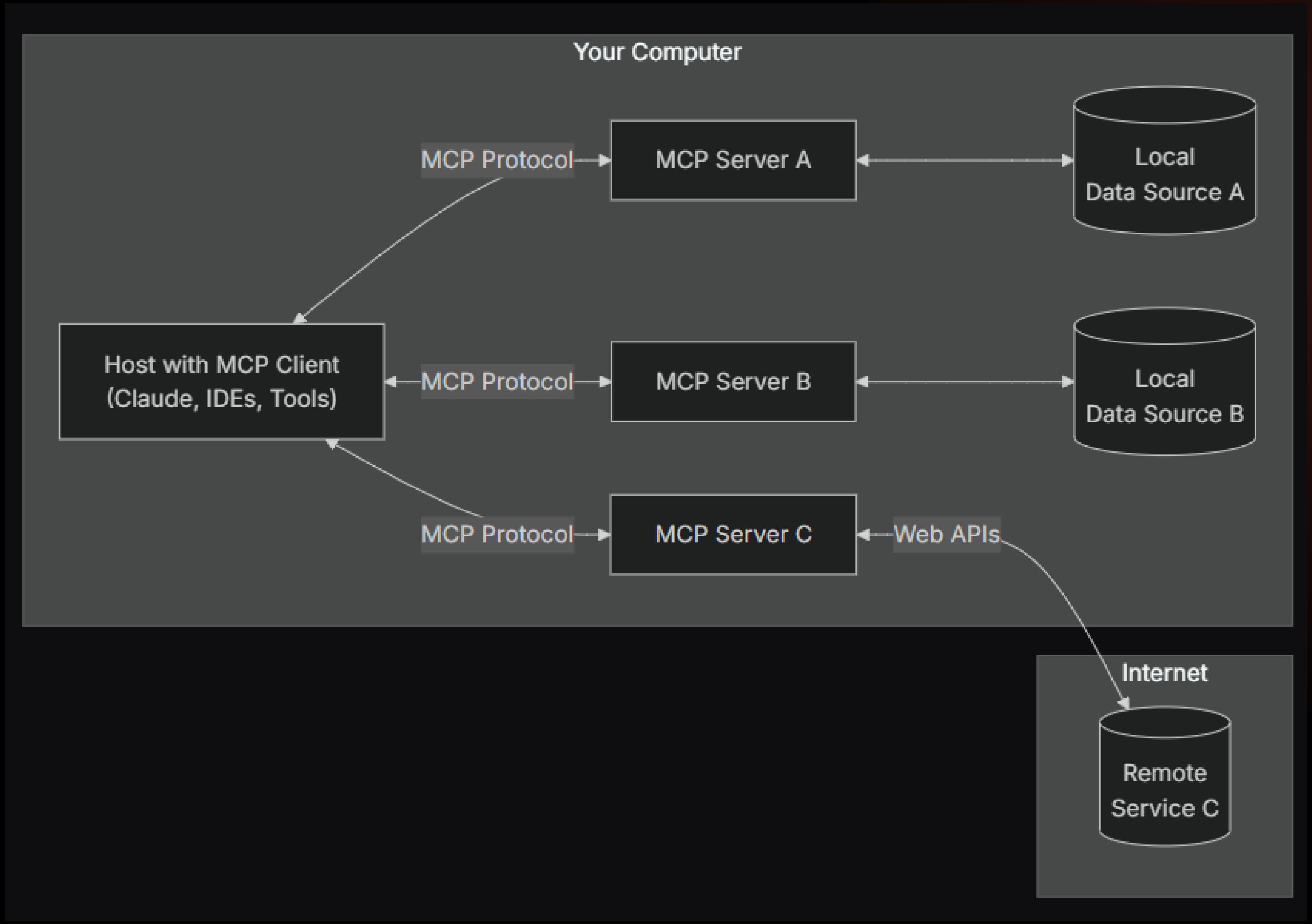
AI model sends a request (e.g., "fetch user profile data").

MCP client forwards the request to the appropriate MCP server.

MCP server retrieves the required data from a database or API.

Response is sent back to the AI model via the MCP client.

Data Flow in MCP



Azure MCP Server

What is Azure MCP Server?

The Azure MCP Server supercharges your agents with Azure context.

Azure MCP Server is Microsoft’s implementation of MCP, designed for seamless connection between AI agents and key Azure services like Azure Storage, Cosmos DB, and more.

Enables you to interact with Azure using natural language such as “List my Azure storage accounts“ or “List containers in my Cosmos DB database”.

Currently in Public Preview.

Key Features of Azure MCP Server

Real-Time Data Fetching: AI assistants can retrieve fresh information from Azure APIs and databases.

Contextual AI Responses: Enhances AI responses by providing accurate, up-to-date information.

Enterprise-Ready Security: Secure and scalable for business applications, including finance, healthcare, and retail

Real World Use Cases

AI-Powered Data Analysis: MCP enables AI models to query Azure Cosmos DB and Azure Data Explorer for real-time insights.

Automated IT Operations: AI agents use MCP to execute Azure CLI commands for infrastructure management.

AI-Driven Monitoring: MCP connects AI models to Azure Monitor for monitoring and observability.

Azure Best Practices: MCP enables secure, production-grade Azure SDK best practices for effective code generation.

Capabilities of Azure MCP Server

Data & AI Search – AI Search for structured retrieval and vector databases	Databases – Cosmos DB (NoSQL) & PostgreSQL for scalable data management	Big Data – Kusto (Azure Data Explorer) for real-time analytics	Storage & Resources – Manage Azure Storage, Resource Groups, and configurations
Logging & Monitoring – Azure Monitor for insights & Log Analytics	Security & Configuration – Key Vault for secure access & App Configuration	Messaging & Automation – Azure Service Bus, CLI & Developer CLI for workflow integration	Best Practices – Enterprise-ready AI with secure deployment

https://github.com/Azure/azure-mcp

Security

Built-In Protections

Subject to Governance: Organisation settings and enterprise policies control Copilot's availability. Any security policies set up for your organisation also apply to Copilot.

Restricted Development Environment: Copilot operates in a sandbox environment with controlled internet access and read-only access to repositories.

Limited Branch Access: Copilot can only create and push to branches starting with **copilot/**. It adheres to branch protections and required checks.

Write Permissions: Copilot responds only to users with write permissions.

Outside Collaborator: Pull requests proposed by Copilot require approval by a user with write permissions before workflows can run.

Compliance Tracking: Copilot's commits are co-authored by the developer who requested the change, ensuring attribution and independent review.

Recommended Practices

Repository Instructions: Add a copilot-instructions.md file to provide Copilot with the necessary context for working successfully in a repository.

Review Code Suggestions: Remember to treat Copilot's suggestions as, suggestions and review in detail.

Secrets Management: Store secrets securely to prevent leaks.

Code Security Features: Enable security features to reduce risks of vulnerabilities.

GitHub Policies: Ensure your GitHub environment is tuned to meet your security needs, configure policies such as branch rulesets, content exclusion, public code suggestion matching etc.

Workshop

Let's turn GitHub Copilot into your Azure sidekick

Prerequisites

1. Install either the stable or Insiders release of VS Code:
   - 💫 Stable release
   - 🔮 Insiders release
2. Install the GitHub Copilot and GitHub Copilot Chat extensions
3. Install Node.js 20 or later
   - Ensure node and npm are in your path
4. Azure CLI
5. Azure Account Setup

Getting Started

1. Clone this repository https://github.com/tw3lveparsecs/azure-mcp-copilot-workshop
2. Follow the setup instructions in **docs/setup-guide.md**
3. Start with the first exercise in the **exercises** directory

Questions